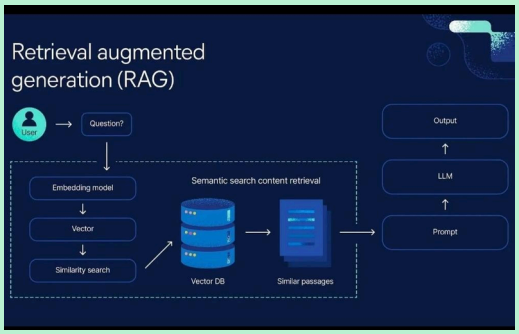
TABBIE AI

TABBIE AI



Introduction

In modern data-driven environments, accessing database insights typically requires knowledge of Structured Query Language (SQL). This creates a significant barrier for non-technical users who need to extract information from databases quickly and .efficiently

To address this challenge, Tabbie AI introduces an intelligent Natural Language to SQL (NL2SQL) middleware that allows users to interact with databases using simple natural language queries. Instead of writing complex SQL commands, users

can ask questions in everyday language and receive .accurate results instantly

The system leverages advanced AI technologies, including Large Language Models (LLMs) and vector-based semantic search, to understand user intent and translate it into executable SQL queries. By combining these technologies with a hybrid memory architecture, Tabbie AI is able to deliver both high accuracy and high perform

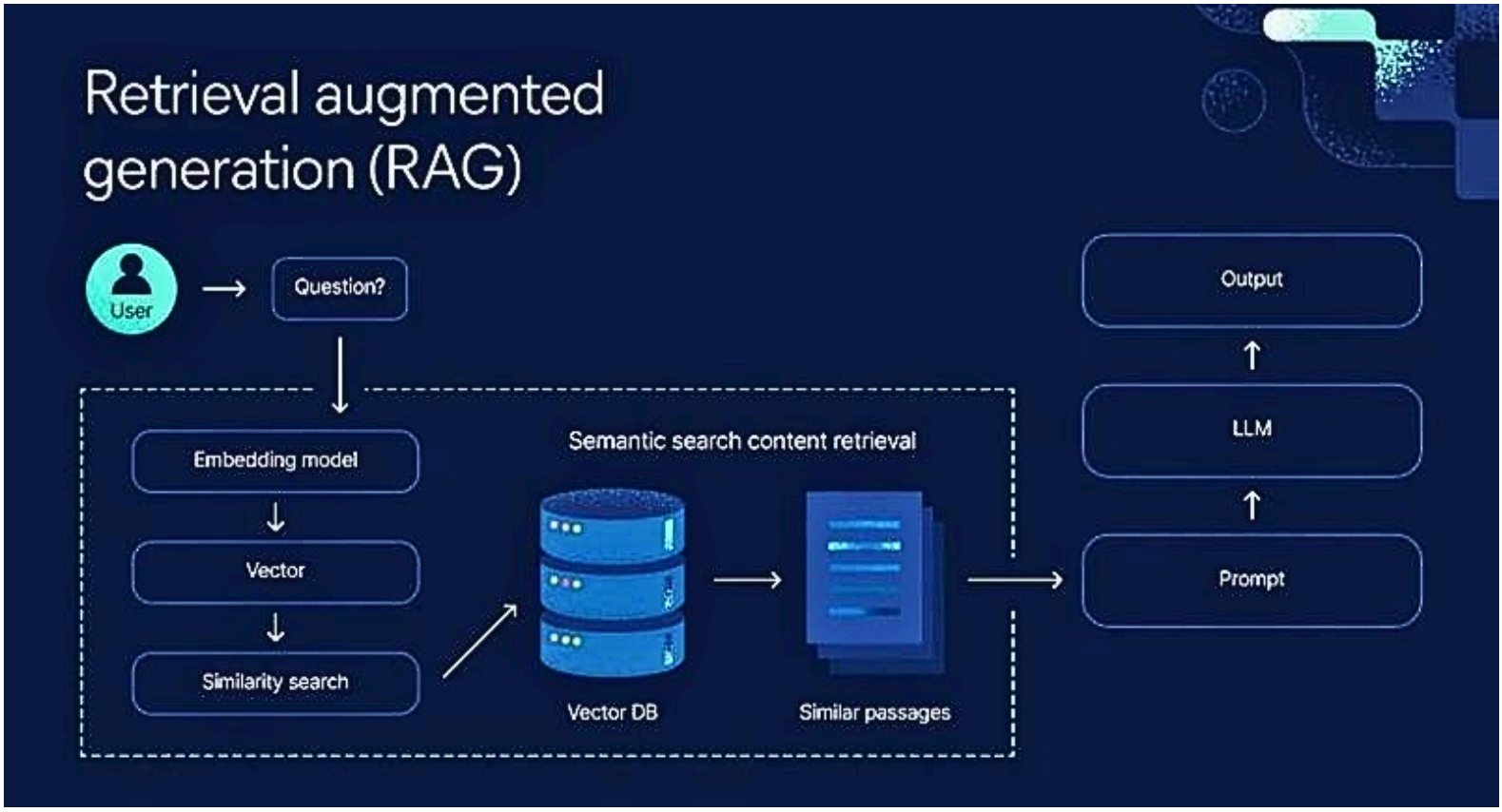


Figure 1: Retrieval-Augmented Generation (RAG) architecture used for semantic search and contextual SQL generation.

System Overview

The Tabbie-SQL Engine is designed as a hybrid AI system that integrates semantic understanding with intelligent query generation. The system processes natural language inputs, analyzes their semantic meaning, and determines the most efficient way to retrieve the requested data from the database.

Rather than relying solely on AI-generated queries, the system incorporates a semantic memory layer that stores previously solved query patterns. This allows Tabbie AI to reuse existing SQL logic when similar queries are detected, significantly reducing computation time and improving system efficiency.

Through this architecture, Tabbie AI provides a balance between speed, scalability, and AI-powered reasoning.

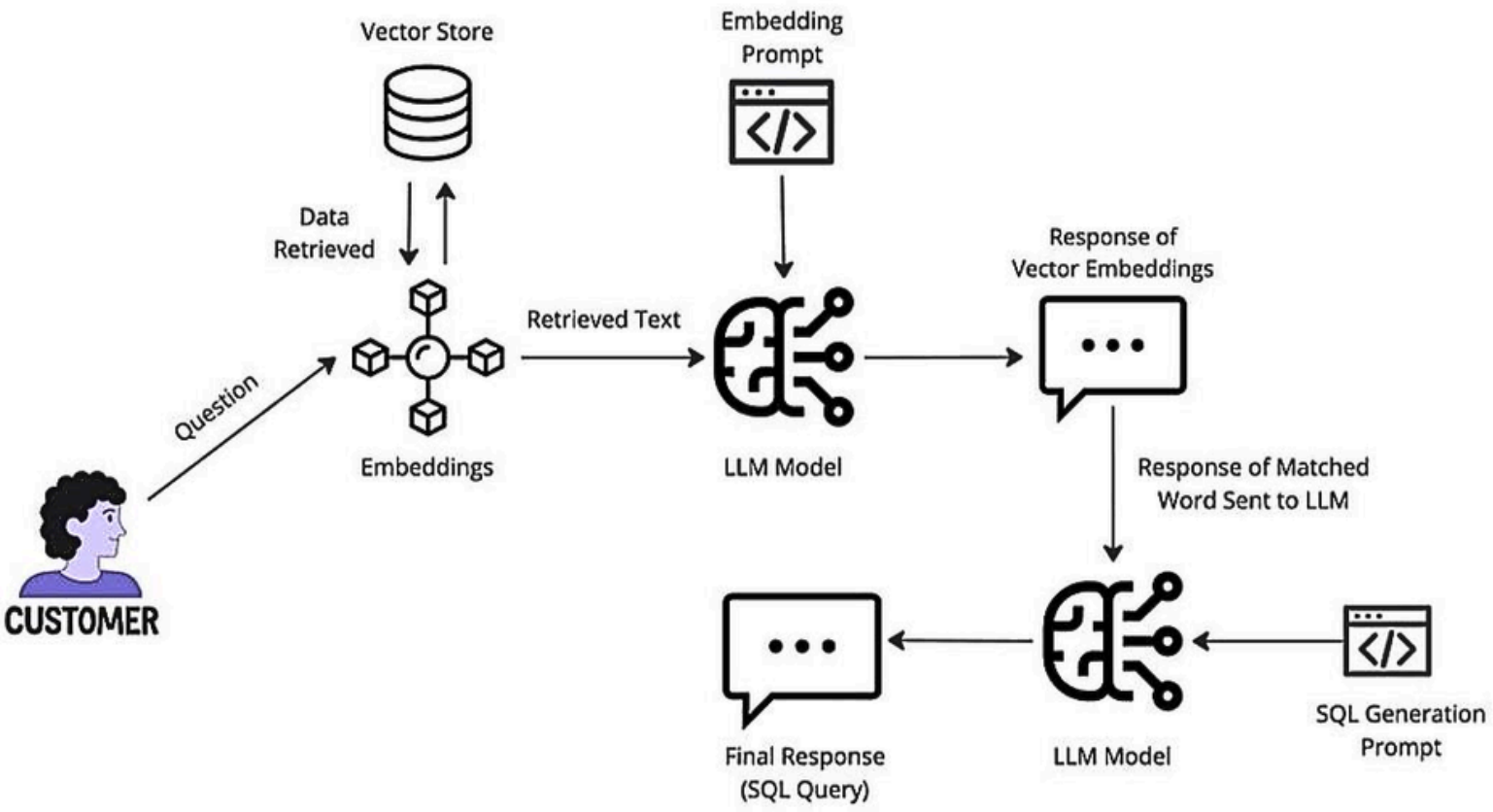


Figure 2: Natural language query processing pipeline from user input to SQL generation.

Query Processing Workflow

To ensure accurate and efficient query translation, the Tabbie-SQL Engine follows a structured multi-stage processing pipeline.

When a user submits a natural language query, the system first converts the input into a semantic vector representation. This allows the system to analyze the contextual meaning of the query rather than relying solely on keyword matching.

Next, the system performs a similarity search within the vector database to identify whether a similar query has been processed before. If a match is found, the stored SQL logic can be reused. Otherwise, the request is forwarded to the AI model to generate a new SQL query.

The following diagram illustrates the complete query processing workflow used in Tabbie AI

System Workflow Overview

User Query

↓

Semantic Embedding

↓

Vector Similarity Search (FAISS)

↓

Logic Guard Check

↓

Decision Engine

↙ ↘

Cached SQL LLM Generation

↓ ↓

Parameter Injection

↓

Database Execution

↓

Results to User

Hybrid Semantic Memory Layer

A core innovation of the Tabbie-SQL Engine is the implementation of a Hybrid Semantic Memory Layer.

This layer enables the system to detect logical similarities between user queries using vector embeddings. Instead of generating a new SQL query for every request, the system compares the semantic representation of the incoming query with previously stored query patterns.

If a match is detected, the engine can reuse the stored SQL logic and simply replace the relevant parameters. This approach significantly improves response time while maintaining the flexibility of AI-generated solutions when necessary.

By integrating semantic search with generative AI, Tabbie AI achieves a powerful balance between intelligent reasoning and computational efficiency.

Chat System and Conversation History

In addition to its query processing capabilities, Tabbie AI provides a persistent chat interface that allows users to interact with the system through conversational sessions.

Each conversation is stored as part of a chat history, enabling users to review previous queries and results. This feature is particularly useful for analytical workflows where users may need to revisit earlier insights.

The system also supports multiple chat sessions, allowing users to create new chats while keeping previous conversations saved and accessible. This design ensures that users can organize their analytical tasks efficiently without losing previous interactions

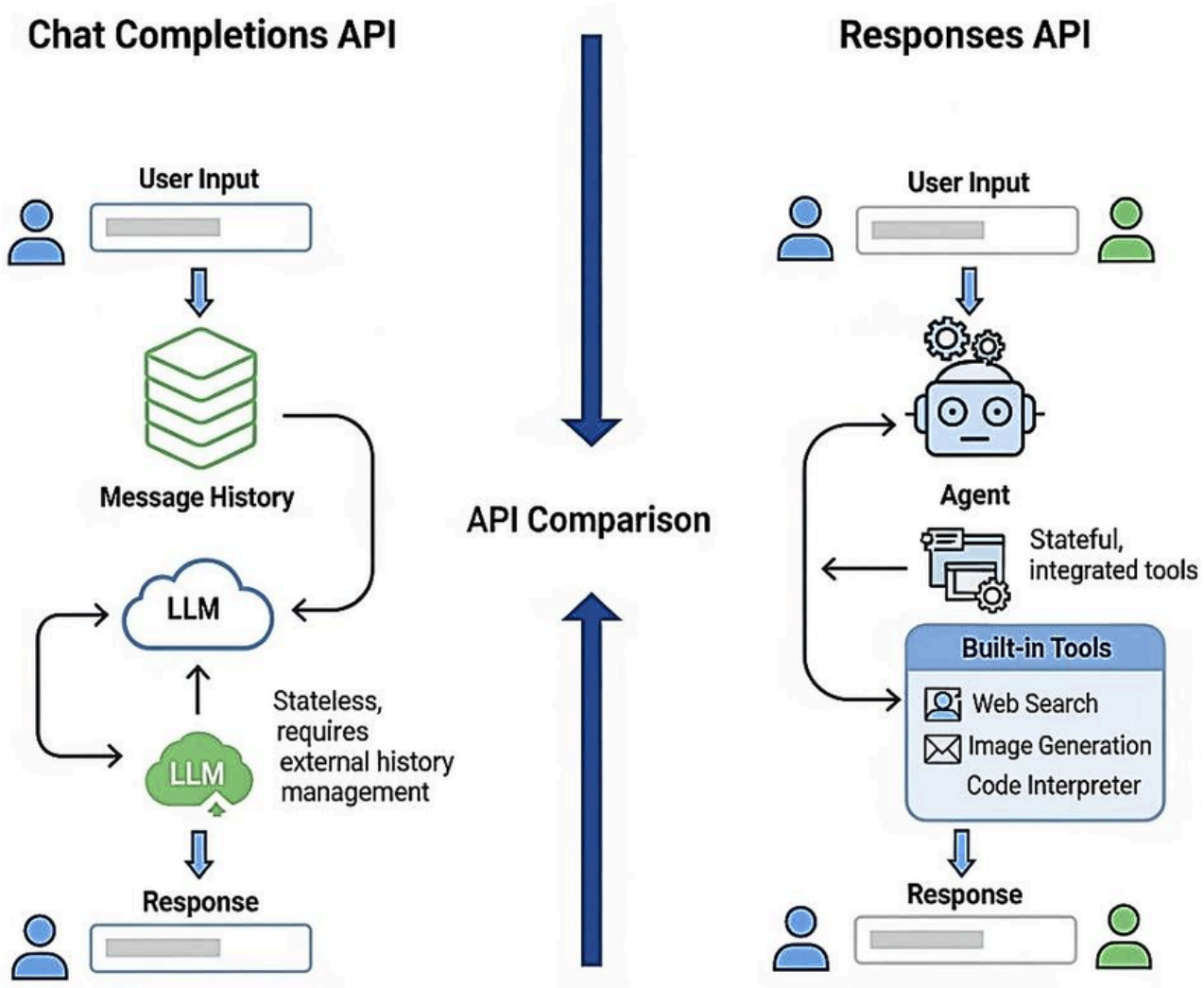


Figure 3: Comparison between chat-based interaction and agent-based response systems.

Technical Stack

The Tabbie-SQL Engine is built using a modern AI and data engineering stack designed to ensure high performance, scalability, and efficient query processing. Each component in the system plays a specific role in enabling natural language queries to be translated into accurate SQL statements.

1- AI Reasoning Model – Google Gemini 2.5

The system uses Google Gemini 2.5 as the primary large language model responsible for understanding natural language queries and generating the corresponding SQL statements. The model analyzes user input, interprets intent, and constructs syntactically correct SQL queries that can be executed on the database.

2- Vector Database – FAISS

To enable semantic similarity search, the system relies on FAISS (Facebook AI Similarity Search). FAISS is an industrial-grade vector database developed by Meta that allows efficient indexing and retrieval of high-dimensional vectors. It enables the system to quickly find previously solved queries that share similar semantic meaning.

3- Embedding Model – HuggingFace all-MiniLM-L6-v2

User queries are converted into vector representations using the HuggingFace all-MiniLM-L6-v2 embedding model. This model transforms text into high-dimensional semantic vectors that capture the contextual meaning of the query, allowing accurate similarity matching within the vector database.

4- Database Connectivity – SQLAlchemy

The system communicates with the database through SQLAlchemy, a powerful Python SQL toolkit and Object Relational Mapper (ORM). SQLAlchemy simplifies database interaction and ensures reliable execution of generated SQL queries.

5- User Interface – Streamlit

The user interface is built using Streamlit, which provides a lightweight and interactive environment for users to submit natural language queries and instantly view the generated results. Streamlit enables rapid development of data-driven web applications with minimal complexity.